

Rekursion Übungen

Fibonacci

Im ersten Semester hast du Fibonacci iterativ implementiert:

```
class Fibonacci {  
  
    static long fibonacci(int n) {  
        if (n <= 1) {  
            return n;  
        }  
  
        long prev1 = 0;  
        long prev2 = 1;  
        long current = 0;  
  
        for (int i = 2; i <= n; i++) {  
            current = prev1 + prev2;  
            prev1 = prev2;  
            prev2 = current;  
        }  
  
        return current;  
    }  
}
```

Die Fibonacci-Folge ist wie folgt definiert:

- $F(0) = 0$
- $F(1) = 1$
- $F(n) = F(n-1) + F(n-2)$ für $n \geq 2$

Schreibe die Methode so um, dass sie **rekursiv** ist.

Teste deine Implementierung mit **nachfolgenden Unit Test**.

Fibonacci: Unit Test

```
class FibonacciTest {  
  
    @Test  
    void fibonacciWith0() { assertEquals(0, Fibonacci.fibonacci(0)); }  
  
    @Test  
    void fibonacciWith1() { assertEquals(1, Fibonacci.fibonacci(1)); }  
  
    @Test  
    void fibonacciWith2() { assertEquals(1, Fibonacci.fibonacci(2)); }  
  
    @Test  
    void fibonacciWith3() { assertEquals(2, Fibonacci.fibonacci(3)); }  
  
    @Test  
    void fibonacciWith4() { assertEquals(3, Fibonacci.fibonacci(4)); }  
  
    @Test  
    void fibonacciWith5() { assertEquals(5, Fibonacci.fibonacci(5)); }  
  
    @Test  
    void fibonacciWith6() { assertEquals(8, Fibonacci.fibonacci(6)); }  
  
    @Test  
    void fibonacciWith7() { assertEquals(13, Fibonacci.fibonacci(7)); }  
  
    @Test  
    void fibonacciWith8() { assertEquals(21, Fibonacci.fibonacci(8)); }  
  
    @Test  
    void fibonacciWith9() { assertEquals(34, Fibonacci.fibonacci(9)); }  
  
    @Test  
    void fibonacciWith10() { assertEquals(55, Fibonacci.fibonacci(10)); }  
}
```

SumList::sum

Erstelle eine Klasse SumList und füge folgende Methode hinzu:

```
static double sum(List<Double> numbers)
```

Die Methode gibt die **Summe aller Elemente** in numbers zurück.

Beispiel: *sum(List.of(1, 2, 3, 4, 5)); // returns 15.0*

Implementiere die Methode **rekursiv**.

Schreibe einen **Unit Test**, um deine Implementation zu testen.

SumListGeneric::sum

Kopiere deine rekursive Lösung von **sum** in eine neue Klasse **SumListGeneric**.

Schreibe deine Lösung **sum** so um, dass sie eine generische Liste **beschränkt auf Numbers** als Parameter erhält (die Methode gibt weiterhin **double** zurück).

Schreibe einen **Unit Test**, um deine Implementation zu testen.

FileSystemElement::elements

Lade den Code für FileSystemElement aus **Moodle** herunter.

Ergänze die Klasse **FileSystemElement** mit folgender Methode:

`abstract` Collection<FileSystemElement> `elements()`;

Implementiere die Methode für die Klasse **File** und **Directory**. Die Methode `elements` gibt eine **unmodifiable Collection** zurück, die das **Element selbst** und **alle enthaltenen Unter-Elemente** enthält.

Prüfe deine Implementierung mit folgenden Testfällen:

```
@Test
void dir1Elements() {
    var actual = dir1.elements();

    var expected = List.of(dir1, file1, dir2, dir3, file2, file3, dir4);
    assertIterableEquals(expected, actual);
}
```

```
@Test
void dir3Elements() {
    var actual = dir3.elements();

    var expected = List.of(dir3, file2, file3);
    assertIterableEquals(expected, actual);
}
```

```
@Test
void file1Elements() {
    var actual = file1.elements();

    var expected = List.of(file1);
    assertIterableEquals(expected, actual);
}
```

MaterialManager 1/2

In der Material- und Produktionswirtschaft werden Stücklisten für die Fertigung von Produkten aus Materialien und Unterprodukten verwendet. Eine Stückliste gibt an wie viel Stück eines bestimmten Materials oder Produktes für die Fertigung eines Produktes benötigt werden.

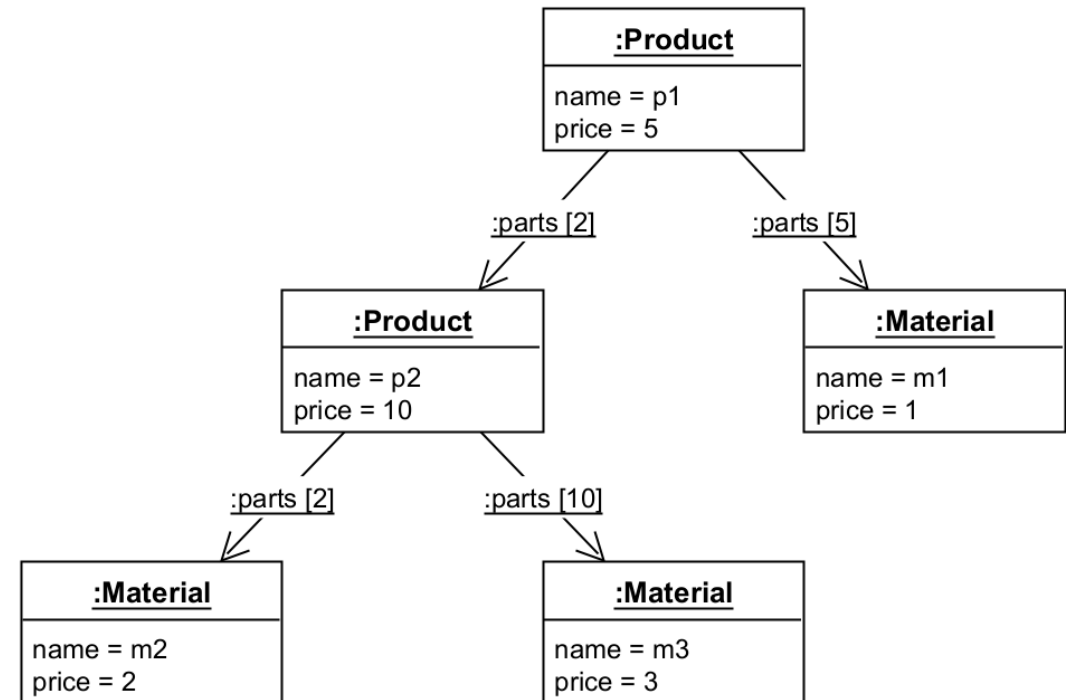
Beispiel:

Wir wollen Produkt p1 herstellen. Dafür benötigen wir 2mal Produkt p2 und 5mal Material m1.

Um p2 herzustellen, benötigen wir wiederum 2mal Material m2 sowie 10mal Material m3.

Welche Materialien muss ein Arbeiter aus dem Lager holen, um ein p1 herzustellen?

Wie hoch sind die Herstellungskosten eines p1?

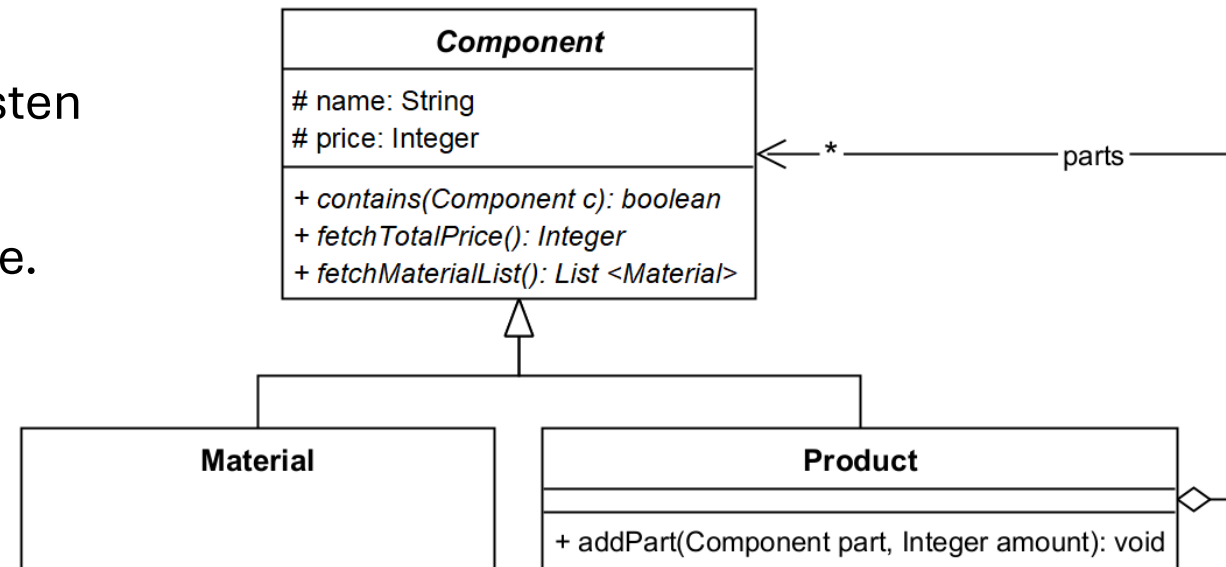


MaterialManager 2/2

1. Übersetze das UML-Klassendiagramm in Java-Code.
2. Erstelle geeignete Konstruktoren.
3. Implementiere die *contains*-Methode.
4. Implementiere die *addPart*-Methode.

Der Parameter *amount* gibt an, wie oft der *part* zum *Product* hinzugefügt werden soll.

5. Implementiere die *fetchTotalPrice*-Methode.
Sie berechnet die gesamten Herstellungskosten eines *Component*.
6. Implementiere die *fetchMaterialList*-Methode.
Sie berechnet eine Liste aller benötigten Materialien für die Herstellung eines *Component*.
7. Nutze folgenden Unit Test zum Testen.



MaterialManager: Unit Test

```
class ComponentTest {
```

```
    private Product p1;  
    private Product p2;  
    private Material m1;  
    private Material m2;  
    private Material m3;
```

```
    @BeforeEach
```

```
    void setUp() {  
        p1 = new Product("p1", 5);  
        p2 = new Product("p2", 10);  
        p1.addPart(p2, 2);  
        m1 = new Material("m1", 1);  
        p1.addPart(m1, 5);  
        m2 = new Material("m2", 2);  
        p2.addPart(m2, 2);  
        m3 = new Material("m3", 3);  
        p2.addPart(m3, 10);  
    }
```

```
    @Test
```

```
    void p1ContainsP2() { assertTrue(p1.contains(p2)); }
```

```
    @Test
```

```
    void p1ContainsM1() { assertTrue(p1.contains(m1)); }
```

```
    @Test
```

```
    void p1ContainsM2() { assertTrue(p1.contains(m2)); }
```

```
    @Test
```

```
    void p1ContainsM3() { assertTrue(p1.contains(m3)); }
```

```
    @Test
```

```
    void p2ContainsM2() { assertTrue(p2.contains(m2)); }
```

```
    @Test
```

```
    void p2ContainsM3() { assertTrue(p2.contains(m3)); }
```

```
    @Test
```

```
    void m1DoesNotContainM1() { assertFalse(m1.contains(m1)); }
```

```
    @Test
```

```
    void p1FetchTotalPrice() { assertEquals(98, p1.fetchTotalPrice()); }
```

```
    @Test
```

```
    void p2FetchTotalPrice() { assertEquals(44, p2.fetchTotalPrice()); }
```

```
    @Test
```

```
    void m1FetchTotalPrice() { assertEquals(1, m1.fetchTotalPrice()); }
```

```
    @Test
```

```
    void m2FetchTotalPrice() { assertEquals(2, m2.fetchTotalPrice()); }
```

```
    @Test
```

```
    void m1FetchMaterialList() { assertEquals(List.of(m1), m1.fetchMaterialList()); }
```

```
    @Test
```

```
    void p1FetchMaterialList() {  
        var expected = Stream.of(  
            Stream.generate(() -> m3).limit(10),  
            Stream.generate(() -> m2).limit(2),  
            Stream.generate(() -> m3).limit(10),  
            Stream.generate(() -> m2).limit(2),  
            Stream.generate(() -> m1).limit(5)  
        ).flatMap(component -> component).toList();
```

```
        assertEqualsIgnoreOrder(expected, p1.fetchMaterialList());  
    }
```

```
    @Test
```

```
    void p2FetchMaterialList() {  
        var expected = Stream.of(  
            Stream.generate(() -> m2).limit(2),  
            Stream.generate(() -> m3).limit(10)  
        ).flatMap(component -> component).toList();
```

```
        assertEqualsIgnoreOrder(expected, p2.fetchMaterialList());  
    }
```

```
    private static <T extends Component> void assertEqualsIgnoreOrder(List<T> expected, List<T> actual) {  
        assertEquals(sorted(expected), sorted(actual));  
    }
```

```
    private static <T extends Component> List<T> sorted(List<T> components) {  
        return components.stream()  
            .sorted(Comparator.comparing(m -> m.name))  
            .sorted(Comparator.comparing(m -> m.price))  
            .toList();  
    }  
}
```

Search Array

Methode `search(int[] array, int target, int index)`: Diese Methode nimmt ein Array, die zu suchende Zahl und den aktuellen Index als Parameter. Die Methode soll rekursiv durch das Array iterieren, bis sie die Zahl findet oder das Ende des Arrays erreicht.

Die Methode soll den Index der Zahl zurückgeben, wenn sie gefunden wird, oder -1, wenn die Zahl nicht im Array enthalten ist.

```
static int search(int[] array, int target, int index)
```